Centro Universitário de Excelência
Sistemas de Informação

MecâniQA — Docker, Docker Compose e Kubernetes

Autores:
Carlos Eduardo Silva Santos
Haniel Pereira Albino da Silva
Ênio Felipe Teixeira Soares
Renato Gomes Souza

Vitória da Conquista, 2026

Agenda

O objetivo dessa apresentação é apresentar a containerização e a orquestração da infraestrutura da MecâniQA Tech.

1. Conteinerização com Docker:
Isolamento da API Java e criação da imagem.

2. Orquestração com Docker Compose:
Integração entre API, MySQL e Redis.

3. Redes e Persistência:
Comunicação por nomes de serviços e volumes persistentes.

4. Kubernete:
Deployments, Pods e Services para gerenciamento do ambiente.

5. Resultados e próximos passos:
Benefícios da arquitetura e evolução para ambientes mais resilientes.

Containerização com Docker

Containerização com Docker

- Aplicação Java empacotada em uma imagem Docker.
- Utilização de Maven + Eclipse Temurin 17 no processo de build.
- Imagem final baseada em Eclipse Temurin 17 JRE Alpine.
- Porta 8080 exposta para acesso à API.
- ENTRYPOINT executa a aplicação Java no container.

Resultado
A aplicação passa a ser executada de forma isolada e reproduzível, sem depender diretamente do ambiente físico.

Docker Compose

Orquestração com Docker Compose

Serviços integrados:

- API Java
- MySQL
- Redis

Configurações principais

- Rede interna porto-velho-net
- Comunicação por nome dos serviços
- Volumes persistentes para MySQL e Redis
- API acessível pela porta 8080
- MySQL: 3306
- Redis: 6379
- healthcheck para verificar o MySQL
- depends_on para controlar a inicialização da API

Principal decisão

A API utiliza mysql e redis como hostnames, evitando IPs fixos e tornando o ambiente reproduzível.

Rede e Persistência

Comunicação interna
A rede:
porto-velho-net

Permite que os containers se comuniquem utilizando os nomes dos serviços, em vez de IPs fixos.

API → MySQL
mysql:3306

API → Redis
redis:6379

Persistência
Foram definidos volumes:
mysql_data
redis_data
Os volumes permitem que os dados permaneçam mesmo quando os containers são reiniciados ou recriados.

Benefício
Separação dos serviços + comunicação interna + preservação dos dados.

Kubernetes

Kubernetes: Gestão e Exposição

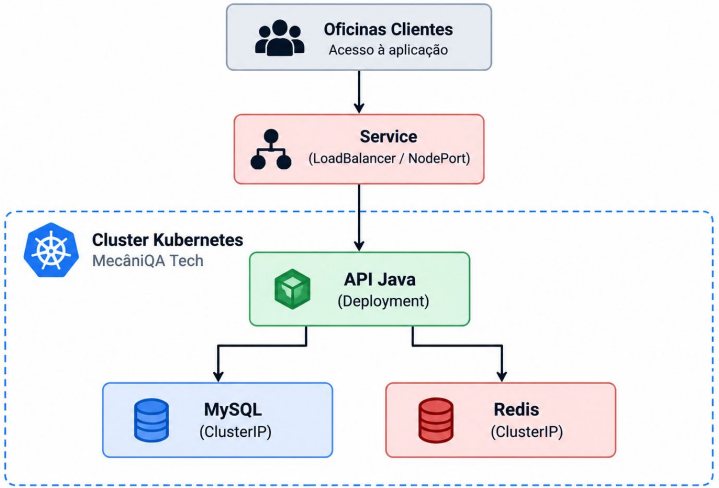
Deployments
- Definem o estado desejado da aplicação.
- Permitem recuperação automática dos Pods em caso de falha.
- Evitam depender da criação manual de Pods.

Services

MySQL → ClusterIP
Comunicação exclusivamente interna.

Redis → ClusterIP
Comunicação exclusivamente interna.

API Java → LoadBalancer / NodePort
Permite acesso externo pelas oficinas clientes.



Considerações finais

- Docker permitiu isolar os componentes da aplicação.
- Docker Compose integrou Java, MySQL e Redis em um único ambiente.
- A rede interna permite comunicação por nomes de serviços.
- Volumes garantem persistência dos dados.
- Kubernetes foi definido para fornecer gestão de estado e recuperação automática.
- Services permitem separar tráfego interno e externo.

Resultado esperado:
Uma infraestrutura mais organizada, reproduzível e preparada para evoluir de um ambiente local para uma arquitetura mais resiliente.

Referências

- MecâniQA Automotive Tech — OAT 1: Docker, Docker-Compose e K8s.
- Centro Universitário de Excelência — Caderno de Guias — OAT 1 — Tutoriais.
- Documentação oficial do Docker.
- Documentação oficial do Docker Compose.
- Documentação oficial do Kubernetes.
- Código-fonte desenvolvido pela equipe.